The Post-API Age

Week 3 · Foundations module · Textbook Chapter 3

Brian C. Keegan, Ph.D.
Associate Professor, Department of Information Science
University of Colorado Boulder

August 31, September 2 & 4, 2026

This week at a glance

The web used to invite us in. That began to change in recent years.

Why did this bargain change? *How* can we keep doing web data science anyway?

Monday | Concepts

- The golden age
- Enclosure, exemption & erosion
- Counter-values, lineages & misconceptions

Wednesday | Notebook lab

- Measure data access and forensic analysis of dead endpoints

Friday | Textbook revisions

- Reading and commenting on issues; first pull requests from the browser

Monday lecture

Read Chapter 03 of the Web Data Science book

Wednesday lab

Complete **Recommended Exercises**: fill in the empty cells, run everything, export to HTML, upload to Canvas by **Sunday 11:59pm**

Friday notebook

You do not need to wait until Friday to begin to submit issues from textbook or notebook, upload the link to your issue or comment by **Sunday 11:59pm**

1. Monday · Concepts

The golden age of web data access

Between roughly 2008 to 2018 the web was a mostly open that was easy(ish) to retrieve and analyze

Websites rendered a mix of static and dynamic content

Social platforms shared data: Twitter in 2006; Facebook in 2010; Reddit's API was free; Google exposed trends

Platforms traded data access for lock-in to their ecosystems, credibility, recruitment

This built my field of computational social science and a generation of data journalism and civic tech

This is the "API age" → we're now in the "post-API age"

They just gave out data for free!

Three pressures

(1) Enclosure, (2) Exemption, (3) Erosion

Why platforms say no

Platforms invoke multiple reasons for restricting researcher access

- **Privacy & safety.** User data can expose identities, enable harassment, or be repurposed in ways users never expected
- **Infrastructure.** Scraping and bulk API access impose real engineering and moderation costs
- **Business.** Data are an asset: competitors, AI companies, and researchers all want something the platform paid to collect
- **Legal risk.** Copyright, privacy law, contracts, and regulatory obligations make broad access risky
- **Control.** Platforms cannot easily distinguish public interest research from spam, surveillance, or commercial extraction

The platform's questions

Who counts as a researcher?

Access to what data?

Under what rules?

How does it help us?

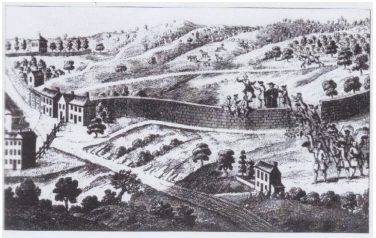
Pressure 1 — Enclosure

Enclosure privatizes a shared resource — traces back to the fencing in of English common land. The data still exist, just on the other side of a paywall.

- **Twitter/X**: a free academic archive (2006–2021) → discontinued; enterprise access starts at $42,000/month
- **Reddit**: free since 2008–2023 → $0.24 per 1,000 calls; 7,000+ subreddit protests and Pushshift cut off

Two drivers:

1. **Independent scrutiny** — researchers and journalists answering uncomfortable questions about polarization, harassment, privacy, financials, *etc*.
2. **AI training data** — uncontaminated user-generated content becomes enormously valuable for training AI models



This land is my land now, it isn't your land anymore

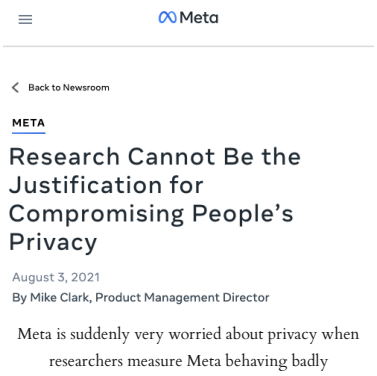
Pressure 2 — Exemption

Exemption: platforms operate without transparency or accountability while surveilling and experimenting on you.

From last week: when "research" appears in a TOS or Privacy Policy, it is generally permission for *them* to do it to *you*

Researchers who want to hold platforms accountable get:

- cease-and-desist letters citing the Computer Fraud and Abuse Act
- NDA'd research collaborations
- data transparency delayed for years
- "privacywashing" and "ethicswashing"

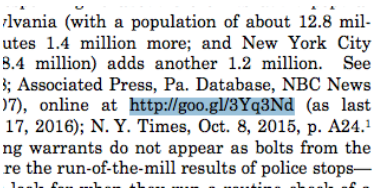


Pressure 3 — Erosion

Erosion is the degradation in technology, practices, and other infrastructure that enables web data science.

- **Links rot.** About half the links in U.S. Supreme Court opinions are dead
- **Platforms churn.** GeoCities vanished; Parler disappears; Twitter became X
- **Sites redesign** and silently break every scraper

For research this is a **reproducibility crisis**: data cannot be re-collected and analyses cannot be re-run → is it still science if it just worked one time for one team?


lvania (with a population of about 12.8 mil-
utes 1.4 million more; and New York City
8.4 million) adds another 1.2 million. See
; Associated Press, Pa. Database, NBC News
)7), online at http://goo.gl/3Yq3Nd (as last
17, 2016); N. Y. Times, Oct. 8, 2015, p. A24.[1]
ng warrants do not appear as bolts from the
re the run-of-the-mill results of police stops—

SCOTUS decisions using broken links

Counter-value 1 — Openness against enclosure

Openness: conduct your own work so that it resists enclosure

Document *how* you collected: endpoints, parameters, dates, and the platform's constraints at collection time

This documentation lets a future researcher understand your dataset after the source changes or vanishes

Practice redundancy: deposit data in durable repositories (Zenodo, Dataverse, etc.), lean on the Internet Archive and Common Crawl, prefer openly licensed sources

Wikipedia as the durable counter-factual — open API, open license, full public dumps

Openness in one habit

Record the date. Preserve your data. Repeat to document changes.

Counter-value 2 — Oversight against exemption

Oversight: consequential systems should be inspected by parties who do *not* own them.

Independent audits of platform algorithms and content moderation; documentation standards that make datasets and methods legible to review

Regulation converts access from a favor into an obligation: EU DSA Article 40 compels vetted-researcher access to very large platforms

This course is oversight

Someone who can retrieve, parse, and analyze web data independently is someone who can hold platforms accountable instead of asking politely

Counter-value 3 — Ownership against erosion

Ownership: prefer arrangements where communities govern the infrastructure their data lives in rather than trusting someone else will keep it running forever

Federated systems on open protocols distribute control: no single acquisition or pricing decision can enclose the whole network

- HTTP, torrents, RSS, Mastodon, Bluesky, *etc.*

Community archives, data cooperatives, and volunteer efforts hold collective traces collectively

The trade

Federation exchanges convenience for control: smaller populations, uneven moderation, more work per query

Myths

Five common beliefs. For each, commit: **true or false?**

1. “The post-API age means APIs are gone.”
2. “If the API closes, I’ll just scrape.”
3. “If data is publicly, collecting it is permitted.”
4. “This only matters for social media research.”
5. “Nothing can be done about it.”

Rules

Write your five answers down first; then we take a hand count, myth by myth. Defend your minority votes — some of these are *almost* true.

Myth busting

1. **APIs are everywhere.** More exist than ever; what is disappearing is *free access for outside observers*, replaced by paid tiers, gated programs, and licensing deals
2. **Scraping is one tool in a portfolio.** Legal risks are changing but non-zero, adversarial site design, poor scalability
3. **Visibility, permission, and ethics are three different questions** A public post's author may never have expected a dataset; a viewable page's ToS may forbid automated collection
4. **Enclosure follows AI-training value** wherever it appears: news archives, code repositories, forums, image libraries. Government data stays open by legal mandate
5. **The environment is contested.** Regulation is creating access obligations, archives are institutionalizing preservation, and federated alternatives are growing

Other professions and disciplines

Public interest data science asks how technical skills can increase the public's capacity to know, scrutinize, and act on consequential institutions and systems.

Retrieving data is never only a technical problem:

- **Access** — who can observe powerful institutions
- **Methods** — what can be made visible and auditable
- **Infrastructure** — whether evidence remains available
- **Practice** — who benefits from our technical work

Other professions have met the problem of powerful actors controlling information the public needs.

Five older traditions

Other flavors of public interest

Journalism — scrutiny
Law — compelled disclosure
Accounting — independent audit
Urban planning — participation
Engineering — public safety

The point

Not one definition of "the public good," but traditions for making power observable, reviewable, and contestable.

Journalism is not just inspiration for public interest data science; it is an older professional technology for working with contested evidence.

FOIA and state records laws turn watchdog work from a polite request into a procedural right: deadlines, exemptions, appeals, and records that agencies did not voluntarily publish.

Verification and provenance are default practices: a claim must trace back to a source, a document, a row, or an interview.

Right of reply treats the target of an investigation as part of the verification process, not as a veto-holder.

Collaborative investigations scale scrutiny beyond any one newsroom.

Data science echo

Journalism's lesson is procedural:

show your work, preserve provenance, invite challenge, publish under pressure

Law contributes procedures for extracting evidence from institutions that have incentives not to disclose it.

Public-records law creates a statutory geometry: request, deadline, exemption, appeal, litigation.

Discovery compels records, testimony, and interrogatory answers that no public API or FOIA request would expose.

Cause lawyering treats a case as a way to change a rule, not merely resolve one dispute.

Sandvig v. Barr matters because it reduced the CFAA chill around audit studies that use bots, paired accounts, or ToS-violating methods.

Data science echo

Legal infrastructure before technical accountability can begin:

requests, complaints, briefs, subpoenas, protective orders.

Accounting contributes the machinery for making claims credible to other institutions.

Audits create an independent record of what was checked, how, and against which standards.

Materiality distinguishes errors that matter from noise: not every discrepancy changes the public claim.

Workpapers make the audit inspectable: evidence, judgment calls, sampling choices, and reviewer sign-off.

Standardized reporting makes one organization's claims comparable to another's.

Data science echo

Datasheets, model cards, replication packages, pinned environments, and audit logs are early versions of inspectability.

Affected publics: technical decisions are also decisions about land, exposure, mobility, housing, and political voice.

Environmental impact statements force agencies to document consequences before irreversible action.

Public comment creates a record, but not necessarily power: consultation can become participation theater.

Stakeholder maps make visible who is in the room, who is affected, who has formal authority, and who has remedy.

Data science echo

Disclose before deployment, map affected parties, document expected harms, and create a record for challenge.

Technical practitioners have public duties that can override duties to clients or employers.

Licensure gives some practitioners standing to refuse unsafe work.

Codes and standards convert past failures into shared constraints on future design.

Ethics canons name the public as a client even when the public did not sign the contract.

Failure analysis treats accidents as institutional evidence with required investigation and disclosure

Data science echo

Data science has no equivalent license to lose, no universally enforced canon, and no mature failure-reporting regime.

Pushback 1 — Sue over the gate

Researchers have challenged the idea that platform terms alone should define the boundary of legitimate inquiry.

- **Sandvig v. Sessions/Barr.** Civil-rights researchers challenged CFAA risk around audit studies using bots, scraping, and tester accounts
- **Van Buren aftermath.** Terms-of-service violations alone are a weaker basis for treating research as "hacking"
- **Strategic target.** The point is reducing the legal chill around public interest auditing

The legal move

Can researchers test platforms the way civil-rights testers test landlords, lenders, or employers?

Pushback 2 — Build around the missing API

Build research capacity outside of platforms with technical infrastructure you control

- **Platform research tools.** Meta Content Library/API, TikTok Research API, controlled-access clean rooms
- **Independent archives.** Internet Archive, Common Crawl, End-of-Term archives, local WARC collections
- **User-side collection.** Data donations, browser instrumentation, panels, screen capture, audit accounts
- **Shared infrastructure.** Replication packages, codebooks, documented missingness, synthetic examples

Major trade-offs with comprehensiveness, reliability, resolution

New access model

Move from a single point of failure to archives + enclaves + donations + audits

Pushback 3 — Turn access into a policy problem

Researchers also push by making platform access a matter of public obligation, not platform generosity.

- **EU: DSA Article 40.** Vetted researchers can seek public or non-public VLOP/VLOSE data
- **Federal proposals.** Researcher requests require ad libraries, moderation stats, and recommender data
- **State transparency laws.** California AB 587 seek disclosure, but 1A challenges narrow what states can compel
- **Public officials on private platforms.** FOIA, state public-records laws, and 1A doctrine can turn official accounts into transparency targets

Public power in private infrastructure still deserves public oversight.

Activity — Platform vs. researcher hidden profile negotiation

Researchers

If your birthday is an even day, you are a naughty researcher trying to get platform data for evil

If your birthday is an odd day, you are a nice researcher trying to get platform data for good

Records — posts, users, comments, ads, algorithms?

Fields — text, timestamps, engagement, deletions?

History — how far back? how often?

Platform operators

If your birthday is an even day, you are a naughty platform trying to protect platform data for evil

If your birthday is an odd day, you are a nice platform trying to protect platform data for good

Privacy — could users be identified or harmed?

Revenue — are you giving away a valuable asset?

Infrastructure — what does bulk access cost?

Competition — could others copy products or models?

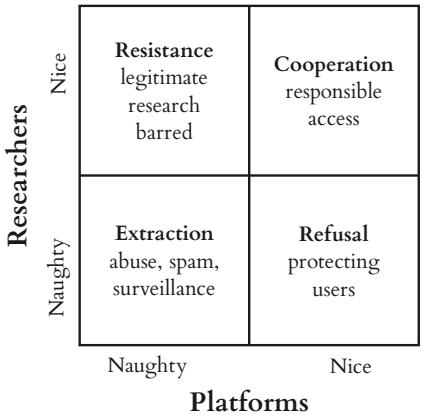
Legal risk — copyright, contracts, regulators, lawsuits?

Enclosure vs. openness; Exemption vs. oversight; Erosion vs. ownership and appeals to public interest

Debrief — What did access cost?

Summarize your deal.

1. What did the researchers originally ask for?
2. What did the platform actually agree to provide?
3. What restriction was hardest to negotiate?
4. What did you trade away to get access?
5. Reproducible by other parties?



Wednesday: measure it, then take it home

Build an access matrix: probe eight live endpoints with no credentials and tabulate who answers, who refuses, and how

Run dead-endpoint forensics: autopsies of Pushshift, CrowdTangle, and Twitter v1.1 — three retirements, three different failure signatures

Take-home: the 7-step audit, new targets

1. Named `User-Agent`
2. Probe the anchor by hand (OpenAlex)
3. Anchor + two public-institution APIs → dated DataFrame
4. Add two institutional endpoints *you* care about
5. Diagnose one refusal, attribute it
6. Autopsy one retired institutional API
7. Write up what you found

None of the take-home's endpoints were shown in class.

2. Wednesday · Notebook Lab

Daily note questions

- **Other resources? —** If you find them, please add links to them for the book (more on Friday!)
- **Need more breakdown —** Last week was a lot, we'll get into a flow every Wednesday and can also use Fridays
- **Highly sensitive "public" data —** Voting registration, property records, criminal records, *etc*.
- **Scrape White House YouTube? —** violation of ToS but permissible under 1A and public officials doctrines
- **Real data negotiations? —** Usually a "wheel war" and uneasy settlements
- **How are GitHub issues resolved? What will happen to our issues? —** We will discuss reviewing, claiming, pull requests, and closing issues on Friday!
- **How often will a site block your scraping? —** Depends how hard you scrape and how sensitive they are: increasing reliance on anti-bot filters and adversarial design
- **How to protect user data? —** Don't generate it in the first place: VPN, Tor, rotating email addresses
- **How to prevent APIs from breaking? —** Need better models in public sphere of supporting for decades
- **What happens if we run out of issues? —** There will always be 40+ things that me or Claude get wrong

Claims you can check — the access matrix

What happens if you show up to a website with no credentials and ask for data?

```
def probe(name, url):
    """Ask one endpoint for data without credentials; describe the answer."""
    try:
        response = requests.get(url, headers=HEADERS, timeout=20)
    except requests.RequestException as error: # no answer is a result too
        return {"api": name, "status": None, "verdict": "unreachable"}
    verdict = ("open" if response.status_code == 200
               else "gated" if response.status_code in (401, 403) else "other")
    return {"api": name, "status": response.status_code, "verdict": verdict,
            "content_type": response.headers.get("content-type", "")}
```

Popular web data sources

Wikipedia, Hacker News, arXiv answer anonymous requests with data

Reddit: status 403, content-type `text/html` a block page served where JSON lived from 2008 to 2023

X: `401 application/problem+json` — a machine-readable refusal

Also fall back to copying the URLs into your web browser

Asking platforms for data these days

Three ways web data dies

How a retired endpoint refuses tells you whether anything can be recovered:

Pushshift answers `403` in JSON. It still runs (for Reddit moderators) but not for you.

CrowdTangle does not resolve at all. No HTTP conversation happens. Enclosed, or destroyed?

Twitter v1.1 parses your request and rejects it (error 215). Alive, but only for paying customers.

HTTP errors

HTTP errors reveal different causes of death: `https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Status`

A guided audit in seven steps

The notebook's **Recommended Exercises** walk you through your own audit of **public-institution APIs** — none of which we probed today. Fill in each empty cell:

1. Define `HEADERS` with your named User-Agent
2. Probe the anchor by hand: **OpenAlex** and its `X-RateLimit` headers
3. Anchor + at least **two** menu endpoints (government, science, museum APIs) → dated DataFrame
4. Add **two** institutional endpoints of your own
5. Diagnose one refusal: `server`, `via`, body — even a `200` that is really a web page
6. Autopsy one retired institutional API: gone, gated, or repriced?
7. Interpret: does "held open by legal mandate" hold?

Effort over perfection — start in class, finish at home, submit what you got working

Graded on good-faith attempt to complete rather than getting everything working

Submitting

Run all cells, then export: *File → Save and Export Notebook As → HTML*.

Upload the `.html` file to Canvas by **Sunday 11:59pm**.

Error flavors

There is some code that is designed to break — these do not need GitHub issues.

Other code may genuinely be broken — experiment to confirm it doesn't work and then search+file issues on GitHub

Additional exercises

The chapter's **Additional Exercises** are optional and worth your time if you are curious:

1. **Extend the autopsy** — two more retired endpoints: gone, gated, or repriced, and who is left to ask?
2. **Timeline** — five API-access events, three platforms, 2008 to today; tag each with its pressure.
3. **Framework application** — 500 words on a platform change *not* in the chapter.
4. **Contingency plan** — a project on one source, plus three fallbacks if it vanishes mid-study.
5. **Public interest audit** — how would current restrictions hit a real API-enabled project?

Advising theses, letters of recommendation, joining research group, *etc.* — email me with findings or file issues if you notice something

3. Friday · Textbook Revisions

Friday: a slower on-ramp

Last week you filed an issue and today you will propose some simple edits.

0. **Keep filing issues** — slides, textbook, Missing Manual
1. **Read the issues** — what is already open across the three repos?
2. **Comment** — claim an issue you will tackle, or sharpen an issue that needs more detail
3. **Edit in the browser** and open a pull request
4. **Attach the issue** to the pull request
5. **Review** — the basics of reading someone else's change

This week's deliverable

The URL of a **new issue** or a **substantive comment** on an existing one, on Canvas by **Sunday 11:59pm**. Pull requests are welcome, not required.

Step 0 — keep filing issues

You can submit issues to any of the three repos this class touches:

- **Textbook** — `github.com/cuinfoscience/Web-Data-Science-Book/issues`
- **Slides & course** — `github.com/cuinfoscience/INFO4617-Fall2026/issues`
- **Missing Manual** — `github.com/cuinfoscience/INFO-Missing-Manual/issues`

Every textbook page now has **Report an issue** and **Edit this page** links in its sidebar. Start from either page you are reading or the repo itself.

Point at the line

On any file in GitHub, click a line number, then **Copy permalink**. Press `y` first to freeze the URL to the current version. An issue that links to a line of code almost always beats descriptions.

Step 1 — read the queue

Before you write anything, read what is already open. An issue page has more on it than a title:

- **Title prefix** — `Broken:` / `Gap:` / `Suggestion:` tells you the type at a glance
- **Labels and assignee** — what kind of work it is, and whether someone already owns it
- **The thread** — reproductions, counter-evidence, scope decisions; read to the bottom before adding to it
- **Linked pull requests** — a fix may already be in flight

Use the search box to find open issues; sort by **Most commented** to find the live ones.

React, don't "+1"

A **thumbs-up reaction** on the issue is how you say "me too." A comment that only says "me too" costs everyone a notification.

Subscribe

Commenting subscribes you to that thread; **Watch** on the repo covers everything. Otherwise you will never see the reply.

Step 2 — comment: claim it or sharpen it

Two kinds of comment earn credit this week.

Claim it. "I'll take this one — planning to …" plus a sentence on your approach. You cannot assign yourself without write access, so the comment *is* the claim; I will set the assignee. Claiming prevents two people fixing the same paragraph, which is the fastest route to a merge conflict.

Sharpen it. Reproduce the problem and say what you saw. Add the missing **Location** or the missing **Evidence**. Propose a scope: "this is really two issues." Ask the author the one question a fixer would need answered.

The grammar: quote with `>`, mention with `@name`, reference with `#12`, and across repos with `cuinfoscience/Web-Data-Science-Book#12`.

Same rubric as an issue

Located, **evidenced**, **actionable**. A comment that adds one of those to someone else's issue is a contribution; "I agree" is not.

Step 3 — the browser is an editor

You do not have write access to these repos, and that is fine — GitHub handles it:

1. Open the chapter's source (**Edit this page**, or find `ch-03-post-api.qmd` in the repo) and click the pencil
2. GitHub says you need to **fork** the repository — click it. A fork is your own copy, under your account, that you *can* edit
3. Make one change. Check the **Preview** tab
4. **Commit changes…** — a one-line title, and a description that says why
5. **Propose changes** — GitHub saves it to a branch in your fork (it will name it `patch-1`)
6. **Compare & pull request** — from `you:patch-1` into `cuinfoscience:main`
7. Fill in the description (next slide), then **Create pull request**

Edit the right file

The `.qmd` is the source. Never the rendered HTML, and never the `.ipynb` — notebooks are regenerated from the chapter and your edit would be overwritten. For slides, the `.tex`.

One change per PR

A PR is a conversation about one change. Twenty typos is one issue listing them — not twenty PRs, and not one PR of twenty.

Step 4 — attach the issue

A pull request that fixes an issue should say so where GitHub can read it. In the PR description:

- `Fixes #12` or `Closes #12` — same repo
- `Fixes cuinfoscience/Web-Data-Science-Book#12` — from another repo

GitHub links the two immediately (see **Development** in the sidebar) and closes the issue automatically when the PR merges. The reporter gets notified; the queue stays honest; the grader finds your work.

The description itself uses the fields you already know from the forms — `Location` · `Problem` · `Why` — plus one new one: `Change`, what this PR does and what you chose not to do.

Draft first

Not sure it is ready? Open it as a **Draft pull request**. Reviewers can see it; nobody can merge it until you mark it ready.

Disclose

If an AI tool helped write or check the change, say so in the description (Appendix B of the book).

Open a classmate's PR and go to **Files changed**. Hover a line, click the **+**, and you are commenting on that exact line. The most useful thing a new reviewer can learn is the **suggestion**: propose the replacement text, and the author applies it with one click.

```
```suggestion
Reddit's API had been free since 2008.
```
```

Finish with **Review changes**: **Comment** (observations), **Approve** (ship it), or **Request changes** (must-fix first). Say which items are must-fix and which are nice-to-have, and end with a verdict.

Who merges

I do. **Merged is not the bar**: you are graded on the diagnosis and the review, not on acceptance.

Tone

Kind, specific, actionable. Assume good faith; name the line; propose the fix.

After the click: checks, conflicts, deploys

Checks. Every textbook PR triggers a render. A green check means the book still builds with your change; a red one means read the log before asking — the failing line is usually named.

Conflicts. "This branch has conflicts that must be resolved" means someone else changed the same lines first. Small PRs and claiming in Step 2 prevent most of it; for a one-liner, GitHub's conflict editor lets you pick a side in the browser. Otherwise, ask.

Deploys. When a PR merges, the book republishes itself. Open the repo's **Actions** tab, watch the run go green, reload the chapter: your sentence is live in about two minutes.

The loop you just closed

Reader → issue → comment → fork → PR → review → merge → live page. Every step of it happened in a browser tab.

A revision menu for Chapter 3

Issues to file or claim — pick one and scope it to a single thing:

- **Read the new material first.** Why Platforms Say No, the three pushbacks (litigate, build, legislate), the negotiation exercise, and the expanded lineages all landed this week. Gaps, overclaims, and unclear passages are likeliest there.
- **Add the missing citations.** *X Corp. v. Bonta*, *Lindke v. Freed*, PATA, and Social Science One are described in prose with no source link.
- **Update the moving numbers.** Twitter/X pricing, the Reddit–Google deal, and DSA Article 40's status "as of mid-2026." Flag what has gone stale.
- **Re-run the audit.** The access matrix and forensics outputs are dated 28 Aug 2026 and *will* drift. Where a row changed, file it with your output.
- **Add a figure.** The chapter still has none: a three-pressures diagram, an API-access timeline (2008–2026), a link-rot chart. Memes welcome.
- **Verify the cross-references.** Ch. 3 leans on many `@sec-` links. Confirm each resolves and is described accurately.

The road from browser to clone

Week 3 (today) — issues, comments, and the browser editor: one file, one change, one PR from a fork.

Week 4 — press `.` on any repo page and GitHub opens VS Code in your browser (github.dev): several files, one commit, still nothing to install.

Week 5 and after — GitHub Desktop or `gh`: clone, branch, commit, push. The full loop on your own machine, with the rendered book checked locally before you propose.

Each rung uses the same four objects — repository, issue, pull request, review — and only changes where the editing happens.

What to submit

Notebook lab: the exported HTML. **Textbook revision**: the URL of your issue or comment. Both on Canvas by **Sunday 11:59pm**.

Next week

Data formats — XML & JSON, the shapes web data actually arrives in.

Key takeaways

1. The open-API era (∼2008–2018) was a **strategic bargain**, not a natural state — and computational social science was built on the trade.
2. **Enclosure** converts collective data into private assets: shutdowns, repricing, and AI-training licensing deals.
3. **Exemption** is the asymmetry of scrutiny — platforms study users freely while blocking those who study platforms.
4. **Erosion** is infrastructural decay, so reproducibility is a problem you must solve *at collection time*.
5. **Access is measurable, not just arguable** — an unauthenticated probe places a platform on the open-to-closed gradient today, and *how* a dead endpoint refuses tells you whether anyone is left to negotiate with.
6. The response is **redundancy**: openness, oversight, ownership — and mastery of many access techniques so no single closure ends your research.

Reading: Textbook Ch. 3. See Freelon (2018); Bruns (2019); Lazer et al. (2020).